

Sistemi Context-Aware, a.a. 2025-2026

Proposte di Progetti

Prof. Marco Di Felice, Prof. Angelo Trotta, Dr. Leonardo Ciabattini
{marco.difelice3, a.trotta, leonardo.ciabattini}@unibo.it

May 20, 2026

1 Proposta 1 – Context-Aware Campus Assistant

Focus: location awareness, geo-fencing, recommendation, spatial data management

Descrizione generale. Si chiede di realizzare una piattaforma context-aware pensata per supportare gli studenti durante la vita universitaria. Il sistema deve suggerire informazioni e servizi utili in base al contesto corrente dell'utente, considerando ad esempio posizione, orario, vicinanza a edifici universitari, servizi disponibili e preferenze personali.

L'applicazione deve essere in grado di mostrare su mappa luoghi rilevanti per lo studente, come aule, biblioteche, sale studio, mense, uffici e altri servizi universitari. Inoltre, deve generare suggerimenti o notifiche contestuali, ad esempio quando l'utente si trova vicino a un edificio rilevante oppure quando un servizio utile è disponibile nelle vicinanze.

Il progetto non deve limitarsi a mostrare una mappa statica. La parte context-aware deve essere esplicita: il sistema deve usare almeno due informazioni di contesto, ad esempio posizione e orario, per produrre un suggerimento diverso a seconda della situazione. Ad esempio, la stessa posizione potrebbe generare un suggerimento diverso al mattino, durante la pausa pranzo o a fine giornata.

- *App mobile.* L'app deve permettere:
 1. Visualizzazione su mappa dei punti di interesse universitari, e.g. aule, biblioteche, sale studio, mense, uffici, segreterie, fermate vicine. I punti di interesse devono essere organizzati in categorie e devono poter essere selezionati dall'utente per visualizzarne i dettagli.
 2. Rilevamento della posizione corrente dell'utente, previa gestione dei permessi di localizzazione. Se non si vuole usare una localizzazione

reale, è accettabile simulare la posizione dell'utente, purché la simulazione sia chiara e modificabile durante la demo.

3. Visualizzazione dei servizi vicini alla posizione dell'utente, ordinati per "rilevanza". La nozione di rilevanza deve essere spiegata nella relazione, anche se basata su una semplice regola, e.g. distanza minima, categoria preferita, servizio aperto in quel momento.
 4. Definizione di un profilo utente minimale, e.g. preferenze su servizi, campus di interesse, mezzi di spostamento preferiti. Il profilo non deve essere complesso, ma deve influenzare almeno una decisione del sistema.
 5. Ricezione di notifiche contestuali basate su posizione e orario, ad esempio "sei vicino a una biblioteca aperta" oppure "la mensa più vicina è a 300 metri". Le notifiche possono essere implementate come notifiche native, messaggi nell'app o eventi visualizzati nella dashboard.
 6. Storico personale delle notifiche o dei suggerimenti ricevuti, in modo da poter mostrare nella demo quali decisioni sono state prese dal sistema e in quale contesto.
- *Back-end.* Il back-end deve:
 1. Gestire un database spaziale contenente i punti di interesse universitari. Ogni elemento deve includere almeno: identificativo, nome, categoria, descrizione, coordinate/geometria, orari o metadati rilevanti. I dati possono essere reali, semplificati o inseriti manualmente.
 2. Fornire API REST per interrogare i punti di interesse vicini a una posizione. Ad esempio, data una coppia latitudine/longitudine e un raggio di ricerca, il sistema deve restituire i servizi presenti nell'area.
 3. Fornire API REST per filtrare i punti di interesse in base a categoria, distanza, orario di apertura o preferenze utente. I filtri devono essere effettivamente usati dall'app o dalla dashboard.
 4. Memorizzare eventi contestuali generati dal sistema, e.g. suggerimenti mostrati, notifiche inviate, PoI selezionati. Ogni evento dovrebbe includere almeno timestamp, utente o sessione, posizione approssimata e motivo del suggerimento.
 5. Implementare una logica base di ranking dei suggerimenti basata su almeno due fattori contestuali, ad esempio distanza e orario. Non è necessario usare machine learning: sono accettabili regole esplicite, purché siano motivate e dimostrate.
 - *Front-end.* La dashboard Web deve:
 1. Consentire la gestione CRUD dei punti di interesse, oppure almeno l'inserimento/modifica di un sottoinsieme significativo di essi.

2. Mostrare una mappa interattiva con i punti di interesse divisi per categoria. La mappa deve permettere di capire visivamente dove si trovano i servizi e quali sono rilevanti per l'utente corrente.
3. Visualizzare statistiche aggregate sull'uso del sistema, e.g. PoI più consultati, categorie più richieste, numero di suggerimenti generati.
4. Consentire il filtro per categoria, area geografica e intervallo temporale, così da analizzare diversi scenari di utilizzo.
5. Mostrare, almeno in forma testuale o tabellare, il motivo per cui un suggerimento è stato generato, e.g. "servizio vicino", "servizio aperto", "categoria preferita".

1.1 Componenti

- **Base 18 pt** Applicazione base:
 - App mobile con visualizzazione della posizione utente e dei punti di interesse universitari su mappa. La posizione può essere reale o simulata, ma deve poter cambiare per mostrare reazioni diverse del sistema.
 - Back-end con database spaziale dei PoI. Il database deve contenere un numero sufficiente di elementi per rendere significativa la ricerca, ad esempio almeno alcune categorie diverse e più punti per categoria.
 - API REST per interrogazione dei PoI vicini. Le API devono essere documentate nella relazione, indicando parametri di input e formato della risposta.
 - Dashboard Web minimale per gestione e visualizzazione dei PoI.
 - Meccanismo base di suggerimento contestuale basato su posizione e orario. Il suggerimento deve essere calcolato dal sistema e non scelto manualmente durante la demo.
- **+2 pt** Geo-fencing e notifiche contestuali:
 - Definizione di aree geografiche associate a edifici o servizi universitari. Le aree possono essere cerchi o poligoni.
 - Rilevamento di eventi di **entry/exit** da geofence, cioè ingresso o uscita dell'utente da un'area rilevante.
 - Invio di notifiche contestuali quando l'utente entra o esce da un'area rilevante. Deve essere chiaro quale regola ha generato la notifica.
- **+2 pt** Personalizzazione dei suggerimenti:
 - Profilo utente con preferenze configurabili. Ad esempio, lo studente può indicare che è interessato a sale studio e biblioteche ma non alle mense.

- Ranking dei PoI basato su preferenze, distanza, orario e categoria. Il ranking deve produrre risultati diversi per utenti o preferenze diverse.
- Storico dei suggerimenti ricevuti e possibilità di feedback dell'utente, e.g. utile/non utile, salvato, ignorato.
- **+2 pt** Integrazione con calendario o agenda:
 - Possibilità di inserire manualmente un calendario di lezioni/eventi. Non è necessario integrare Google Calendar o servizi esterni: è sufficiente una semplice agenda interna.
 - Suggerimenti contestuali rispetto al prossimo impegno, e.g. percorso verso l'aula, servizi vicini prima della lezione, avviso se l'utente è lontano dall'aula poco prima dell'inizio.
- **+2 pt** Analytics spaziale:
 - Heatmap dei punti di interesse più consultati o delle aree in cui vengono generati più suggerimenti.
 - Analisi delle aree più frequentate o più rilevanti. L'analisi può essere semplice, ma deve essere basata sui dati raccolti dal sistema.
 - Grafici temporali sull'utilizzo dei servizi, ad esempio numero di suggerimenti per ora o categorie più richieste durante la giornata.
- **+2 pt** Privacy location-aware:
 - Integrazione di almeno una tecnica di protezione della privacy della posizione tra quelle discusse a lezione, e.g. perturbazione, cloaking, dummy location.
 - Valutazione dell'impatto della tecnica su *Privacy Perturbation* e *Quality of Service*, mediante grafici. Ad esempio, si può mostrare come l'aumento della perturbazione migliori la privacy ma riduca la precisione dei suggerimenti.
- **+2 pt** Supporto offline:
 - Caching locale dei PoI in una determinata area, ad esempio i servizi del campus selezionato.
 - Visualizzazione della mappa e dei servizi salvati anche in assenza di connessione. Durante la demo deve essere mostrabile almeno un caso in cui l'app funziona con dati precedentemente salvati.
- **+2 pt** Containerizzazione con Docker Compose:
 - Containerizzazione di back-end, database e front-end.
 - Script Docker Compose per avviare l'intera piattaforma con un unico comando o con pochi comandi chiaramente documentati.

1.2 Tecnologie consigliate

- **App mobile:** Android nativo, Flutter, React Native o framework a scelta. *L'interfaccia dell'app può essere minimale e non è oggetto di valutazione.*
- **Back-end:** Node.js, Python, Java o framework a scelta.
- **Database:** Postgres/PostGIS per la gestione dei dati spaziali. In alternativa, per prototipi semplici, è possibile usare un database non spaziale o file GeoJSON, purché le operazioni spaziali richieste siano implementate chiaramente.
- **Front-end:** OpenLayers o Leaflet per la mappa; librerie a scelta per grafici.
- **Containerizzazione:** Docker e Docker Compose.

E' possibile utilizzare un qualsiasi linguaggio di programmazione o librerie a scelta dello studente.